Handoff 04: firmware build, fuses and flashing

For: Claude Code in VS Code with PlatformIO. Matt connects the programmer and the board.
Before you start: the main board is assembled and has passed VERIFY E1–E4 (handoff 05, step 3).

1. Build and test on the laptop:

   ```
   cd firmware && pio test -e native && pio run
   ```

   The host tests must pass (32 tests), and the ATtiny3216 build must succeed.
2. **Programmer on J4.** Pins: 1 = VREF (VSYS through 1 kΩ, *sense only*), 2 = UPDI, 3 = TX, 4 = GND. The square pad is pin 1. J4 is not fitted on the board: hold a 4-pin 2.54 mm header in the holes, or solder one temporarily.
   - **Power the board from its own battery (slide switch ON).** USB-C also works, but the battery is better with a 3.3 V adapter (see below). Never power it from the programmer: J4 pin 1 cannot feed the board safely, and it would back-feed the LiPo through 1 k (round1 M3).
   - **SerialUPDI (recommended):** a 3.3 V USB-UART. Wire UART TX → 470 Ω → UPDI, UART RX → UPDI directly, GND ↔ GND. Leave the adapter's VCC unconnected. SerialUPDI doesn't use VREF at all.
   - **MPLAB SNAP / PICkit 4:** set `upload_protocol = snap_updi` (avrdude) in the env, or use MPLAB IPE. It senses the target voltage through J4.1's 1 kΩ. That should work, but SNAP's sense current is unspecified (round2 note). An Atmel-ICE draws up to 1 mA there and can fail at low battery.
   - Prefer the battery (3.7–4.2 V) over USB (VSYS ~4.0 V) when using a 3.3 V adapter: the UPDI levels then sit closer together.
3. **Fuses, once per chip (FW-09).** `upload` does not write fuses: without this step BOD stays off and the saved default is erased on every reflash.

   ```
   pio run -e attiny3216 -t fuses --upload-port /dev/ttyUSB0
   ```

   This sets BOD 2.6 V (enabled while running, off in sleep: fuse BODCFG 0x54), EESAVE on, and a 10 MHz clock.
4. Flash:

   ```
   pio run -e attiny3216 -t upload --upload-port /dev/ttyUSB0
   ```

5. **Debug log:** flash `-e attiny3216_debug` instead. Connect a 3.3 V USB-UART RX to J4 pin 3 at 115200 baud. Once a second it prints `T=` (°C, from the MCP9800; −99 = no reading), `V=` (VDD mV), mode, level, the thermal cap, `lp=` (the LP5012's DEVICE_CONFIG1 read-back: **1C** when the driver is configured, `--` when it does not answer) and `ts=` (1 = thermostat limits written and verified). A `BOD OFF` line at boot means step 3 was skipped. LED board v0.2 addresses: LP5012 **0x14**, MCP9800A0 **0x48**. If `lp=--` with the board powered, check U1's joints (VERIFY E16) and the cable order (G V D C). Disconnect the adapter before you switch the lamp off: its RX pull-up can back-power the MCU (N6).
6. **VDD calibration check (VERIFY F3).** With the debug build, compare `V=` against a meter on TP3 (VSYS) at about 3.5, 3.8 and 4.1 V. Use a bench supply on J2, or a cell at those states. If the error is more than ±50 mV, ask Claude to add a calibration factor to `config.h` (`vdd_mv()` scales by it).

7. **Bench fault builds (VERIFY T5a/T5b/E17 only):** `-e bench_t_nack` fakes a dead sensor, `-e bench_t_stuck` fakes a frozen 25 °C reading, and `-e bench_tstat_low` sets the hardware thermostat to 30/28 °C so a warm finger on U2 trips it. **Reflash the release build afterwards.**
8. **Mode tuning:** after the LEDs are fitted, measure the whites with a spectrometer, or roughly with a phone colour meter (VERIFY O6/O7). Put the measured xy and lumens into `LED` in `firmware/tools/gen_modes.py`, then run `python3 firmware/tools/gen_modes.py` and rebuild. Never edit `include/modes_table.h` or `include/board_map.h` by hand: both are generated.

Done when

- The lamp boots into Soft white 3000 K, fading in.
- MAIN cycles the colours. UP/DOWN step the brightness and auto-repeat when held.
- Holding MAIN for 2 s saves the default; you see two dips.
- The debug log shows a sane VDD (within ±50 mV of the meter) and temperature.
- The fuses are written (no `BOD OFF` line).
- The release build is flashed.